



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Faculty of Science and Technology

Department of Computer Science

CSC 3312: SOFTWARE ENGINEERING

Spring 2025-2026

Project Title: GeoMart

Supervised By

Saikat Baul

Submitted By

<i>Group: 6</i>		<i>Section: R</i>
SL	Student Name	Student ID
1.	Md Bony Amin	24-56311-1
2.	Niloy Bormon	24-56200-1
3.	Sazzad Hossain Rabby	23-54859-3
4.	Abdullah Ar Rafi	23-53776-3
5.	Sumaiya Elma Eshara	23-53775-3

Table of Contents

1. INTRODUCTION	3
1.1 Background to the Problem	3
1.1.1 Target Group of Users.....	3
1.2 Selection of Process Model.....	4
2. SOFTWARE REQUIREMENTS SPECIFICATIONS	7
2.1 User Story Table	7
2.2 Requirements Traceability Matrix	11
2.2.1 Functional Requirements.....	11
2.2.2 Non-Functional Requirements.....	17
3. SOFTWARE DESIGN.....	20
3.1 System Design	20
3.1.1 Use Case Diagram	20
3.1.2 Class Diagram.....	21
3.1.3 Activity Diagram	22
3.2 Wireframe Design	23
4. GIT WORKFLOW	26
5. SOFTWARE TESTING.....	30
6. CONCLUSION.....	38

1. INTRODUCTION

1.1 Background to the Problem

Today, most people buy things online using big websites. The problem is that when we order something, it usually takes 2 or 3 days to arrive. These products often come from a warehouse far away. However, many times the exact product you want is already available in a shop right in your own neighborhood. The issue is that customers do not know which nearby shops have the product in stock, so they order from far away instead. Small and medium-sized local shops struggle because they do not have a way to show their products online. They lose customers to big websites even though they are closer to the buyer. Also, delivering items from far away is expensive and slow.

GeoMart solves this by connecting local shops directly to the people living near them. It creates a system where a buyer can search for a product and see all the shops in their area that have it. Because the shops are nearby, the delivery can happen on the same day, sometimes in just an hour. This helps local businesses grow and helps customers get what they need much faster.

GeoMart serves multiple types of users who interact with the platform for different purposes.

1.1.1 Target Group of Users

There are three main groups of people who will use **GeoMart**:

1. The Buyers (Customers)

Buyers are local residents who want to purchase products quickly without waiting days for traditional shipping. They need a simple way to see exactly what neighborhood shops have in stock through a digital storefront. **GeoMart** helps them by providing a live map of nearby stores and the convenience of having their items delivered on the same day by a local rider or store staff.

2. The Sellers (Shop Owners)

Sellers are neighborhood store owners, ranging from small shops to larger outlets, who want to grow their business by selling online. They need an easy-to-use system to list their inventory and manage orders from people living nearby. The platform helps them reach local customers instantly and gives them the choice to either deliver orders themselves during off-hours or use the app's rider network for convenience.

3. The Riders (Delivery People)

Riders are independent individuals or local delivery services looking for flexible work within their own community. They need a straightforward platform that alerts them to nearby delivery jobs that can be finished quickly. Because the distances between the shop and the buyer are short, riders can complete multiple deliveries in a single day and earn money based on the distance traveled.

1.2 Selection of Process Model

For the **GeoMart** platform, we have selected **Scrum (Agile)** as our software development process model. Scrum is an adaptive and iterative framework that organizes development into short, time-boxed cycles called Sprints, enabling the team to deliver work in small, functional increments and incorporate feedback continuously. This model is highly suitable for our project because GeoMart is a multi-tiered system comprising three distinct but highly interconnected user roles (Buyers, Sellers, and Riders) and complex modules (location-based shop mapping, live inventory syncing, real-time order processing, and delivery tracking). These characteristics benefit immensely from incremental planning, frequent review, and the ability to refine features based on the practical realities of hyperlocal e-commerce and fast-paced delivery workflows.

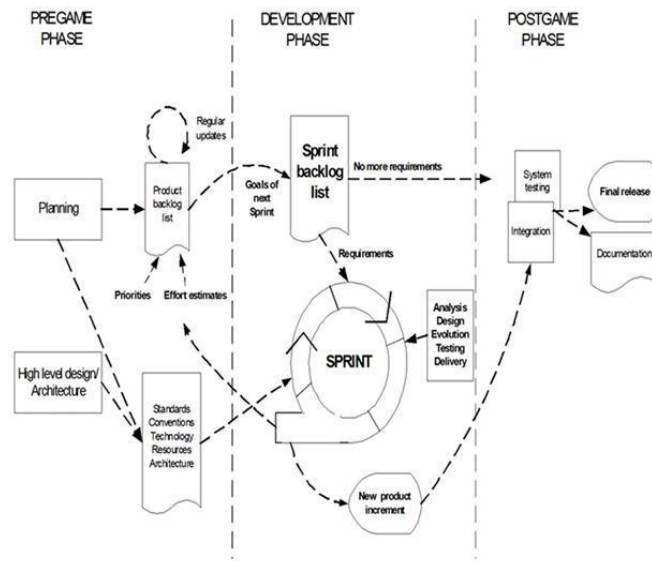


Fig 1: Scrum (Agile) Process Model for GeoMart System Development

The Scrum process is commonly described using three phases: Pre-game, Development (Game), and Post-game.

1) Pre-game Phase (Planning + High-Level Design/Architecture): In this initial phase, the team performs project planning and prepares a Product Backlog, which is a prioritized list of all required features and user stories. For GeoMart, this backlog includes items such as "Search for products in nearby shops," "List and update store inventory," "Accept a delivery ping," and "Track rider location in real-time." Concurrently, the team establishes the high-level system architecture, defining core components (e.g., location-based geo-querying, the three-tier authentication system, and the real-time database schema for order states), key user role interactions, and overall system flow. Effort estimates and business priorities are assigned to backlog items to facilitate effective sprint planning.

2) Development Phase (Sprint Cycle): During the development phase, the team conducts a series of Sprints. At the start of each Sprint, items are selected from the Product Backlog to form a Sprint Backlog, representing a committed set of goals for that iteration. Each Sprint then follows a cycle of detailed analysis, design, implementation,

testing, and review, culminating in a potentially shippable product increment. In the context of our project, a completed increment may be a cohesive set of deliverables such as updated sections of the Software Requirements Specification (SRS), detailed UI/UX wireframes for a specific module (e.g., the Seller's inventory dashboard or the Rider's map interface), corresponding system diagrams (Use Case, Class, Sequence), and a suite of validated test cases verifying the interaction between the local shop and the customer.

3) Post-game Phase (Integration + System Testing + Documentation + Release):

After sufficient increments have been developed and reviewed, the project enters the Post-game phase. Here, all individual modules (the Buyer app, Seller portal, and Rider interface) and their associated documentation artifacts are integrated into a cohesive whole. Comprehensive system testing is performed to ensure the three tiers interact flawlessly—verifying that when a Buyer places an order, the local Seller receives it instantly, and upon processing, a nearby Rider is successfully pinged for same-day delivery. Finally, all project documentation is consolidated, reviewed for consistency, and prepared for final submission and presentation.



Fig 2: Agile (Scrum) Process Flow Diagram for GeoMart

Why Scrum is the Most Appropriate Model for GeoMart?

Scrum is the ideal framework for developing GeoMart for the following reasons:

Incremental Delivery of Complex, Multi-Tier Modules: GeoMart is fundamentally a three-part ecosystem (Customer App, Shop Owner Dashboard, Delivery App). Scrum allows us to logically decompose the system and build it out one core workflow at a time. For example, we can use early sprints to solidify the Buyer-to-Seller order mechanism before introducing the complexities of Rider GPS matching. This reduces integration risk and ensures the core goal-connecting locals to nearby inventory-is functional early on.

Adaptability to Real-World Variables: Hyperlocal delivery involves unpredictable real-world factors, such as inaccurate map pins, varying shop operating hours, or rider availability edge cases. Scrum's regular sprint reviews and backlog grooming sessions allow us to test these workflows practically and refine the UI/UX and business logic. If we

discover that local shops need a simpler way to toggle "out of stock" items quickly, we can adapt the requirements in the very next sprint.

Effective Management of Team Roles and Tasks: Scrum's structured framework is highly effective for coordinating a team working on interconnected data streams. It allows for clear task division-such as assigning one member to focus on the Rider's geolocation logic while another builds the Buyer's product search algorithm. Daily stand-ups ensure that the team remains aligned, preventing data mismatches between the three user interfaces and rapidly resolving blockers.

Superior Fit Over Sequential Models: A strictly sequential model like Waterfall assumes that all user interactions and edge cases are perfectly understood upfront. Given the dynamic nature of a live marketplace like GeoMart, discovering nuanced requirements during development is inevitable (e.g., determining the exact geographical radius for "nearby" shops or how to handle a rider canceling mid-delivery). A flexible Agile approach allows us to evolve the SRS, system diagrams, and prototypes progressively without derailing the entire project timeline.

2. SOFTWARE REQUIREMENTS SPECIFICATIONS

2.1 User Story Table

AS A/AN	I want to	So that	Acceptance Criteria
The Buyer	Search for products in nearby shops	I can find items quickly	Nearby shops with products are displayed
	View products on a map	I can see closest shops	Map shows shop locations and distance
	Compare prices	I can choose the best option	Prices from multiple shops are visible
	See real-time stock	I avoid unavailable items	Stock status is updated (Available/Out of Stock)
	Order products online	I can purchase products successfully	Order is successfully placed and confirmed
	Choose delivery or pickup	I get flexibility	Option available during checkout
	Track orders live	I know delivery status	Real-time tracking is shown
	Get same-day delivery	I receive items faster	Delivery completed within hours
	Use multiple payment methods	I can pay easily	Supports cash, mobile banking, card
	Rate shops and riders	I can share feedback	Rating system works after delivery
	Filter products by category	I can find items easily	Filter option shows correct categories

	Sort products by price or distance	I can choose better options	Sorting works correctly
	Save favorite shops	I can reorder quickly	Favorite list is saved
	View product reviews	I can make better decisions	Reviews are visible
	Cancel orders	I can manage mistakes	Order cancels successfully
	Chat with sellers	I can ask about products	Messaging works properly
	Receive order notifications	I can stay updated	Notifications are received instantly
	View delivery time estimates	I can plan better	Estimated time is shown
	Apply discount coupons	I can save money	Discount applies correctly
	Reorder previous items	I can save time	Previous orders are accessible
	Track payment history	I can manage expenses	Payment history is stored
	Change delivery address	I can receive orders anywhere	Address updates successfully
The Seller	Register shop	I can sell online	Shop account is created
	Add/manage products	Customers can see inventory	Products can be added/edited/deleted
	Update stock	Customers can see correct data	Stock updates instantly
	Receive order notifications	I don't miss orders	Notifications appear instantly

	Accept/reject orders	I can control sales	Order status updates properly
	Set prices	I can manage profit	Price updates are saved
	Choose delivery method	I control logistics	Option for self or rider delivery
	Track orders	I can manage workflow	Orders visible in dashboard
	View sales reports	I can track performance	Reports show earnings
	Communicate with buyers	I can clarify details	Messaging system works
	Upload product images	Customers see items clearly	Images upload successfully
	Set product categories	Items are organized	Categories are assigned correctly
	Manage discounts	I can attract customers	Discounts apply properly
	Mark shop open/closed	Customers know availability	Status updates correctly
	View pending orders	I can prioritize work	Orders are listed properly
	Receive payment reports	I can track earnings	Reports are generated correctly
	Manage delivery time	I can meet expectations	Delivery time is updated
	Block fake users	I avoid fraud	Users are blocked successfully

	View customer feedback	I improve service	Feedback is visible
	Manage multiple branches	I expand business	Multiple shops are handled
	Export sales data	I analyze offline	Data exports correctly
	Set minimum order amount	I manage costs	Limit is applied properly
The Rider	Register as rider	I can start working	Rider account created
	Get nearby delivery requests	I reduce travel time	Nearby jobs are shown
	Accept/reject jobs	I control workload	Job status updates
	View pickup/drop locations	I know where to go	Locations clearly displayed
	Navigate via map	I reach destination easily	Navigation is integrated
	Deliver multiple orders	I earn more	Multiple deliveries allowed
	Update delivery status	Users get updates	Status updates in real-time
	Track earnings	I monitor income	Earnings dashboard available
	Receive payments	I get paid properly	Payments processed correctly
	Rate buyers/sellers	I give feedback	Rating system available
	Set availability status	I control working hours	Status updates correctly

	View daily delivery history	I track work	History is displayed
	Receive job notifications	I don't miss jobs	Notifications work instantly
	Cancel accepted jobs	I handle emergencies	Job cancels successfully
	View route optimization	I save time	Best route is shown
	Contact buyers/sellers	I solve issues	Communication works
	See estimated earnings	I decide better	Earnings are shown before accept
	Report delivery problems	System handles issues	Reports are submitted
	Track fuel cost	I manage expenses	Cost tracking is available
	Receive bonuses	I stay motivated	Bonus system works
	Verify delivery via OTP	Delivery is secure	OTP verification works
	Pause notifications	I can rest	Notifications can be disabled

2.2 Requirements Traceability Matrix

2.2.1 Functional Requirements

1.1.0	Registration
1.1.1	Click "Sign Up" button
1.1.2	Select role (Buyer / Seller / Rider)

1.1.3	Enter user name / shop name
1.1.4	Enter phone number
1.1.5	Enter email
1.1.6	Enter password
1.1.7	Confirm password
1.1.8	Click "Next" button
1.1.9	Receive OTP
1.1.10	Enter OTP
1.1.11	Submit registration
1.1.12	Show "Registration Successful"
1.2.0	Login
1.2.1	Enter username / email
1.2.2	Enter password
1.2.3	Provide forgot password option
1.2.3.1	Submit email for password recovery
1.2.3.2	Receive reset link / OTP
1.2.3.3	Reset password
1.2.4	Submit login credentials

1.2.5	System validates information
1.2.6	If wrong → show error message
1.2.7	If correct → open dashboard
1.3.0	Dashboard
1.3.1	Side menu
1.3.1.1	Dashboard button
1.3.1.2	View profile button
1.3.1.2.1	Show name
1.3.1.2.2	Show email
1.3.1.3	Order history button
1.3.1.4	Notification panel
1.3.2	Display nearby shops
1.3.3	Show featured products
1.4.0	Search Product
1.4.1	Click search field
1.4.2	Enter product name
1.4.3	Show nearby shops
1.4.3.1	Display shop name

1.4.3.2	Display distance
1.4.4	Apply filters
1.4.4.1	Filter by category
1.4.4.2	Filter by price
1.4.5	Sort results
1.4.5.1	Sort by price
1.4.5.2	Sort by distance
1.5.0	Product Details
1.5.1	Click product
1.5.2	Show product name
1.5.3	Show price
1.5.4	Show stock status
1.5.5	Show reviews
1.5.6	Show shop location
1.6.0	Order System
1.6.1	Add to cart
1.6.2	View cart
1.6.3	Checkout

1.6.3.1	Select delivery option
1.6.3.2	Select payment method
1.6.4	Confirm order
1.6.5	Order placed successfully
1.7.0	Order Tracking
1.7.1	Click track order
1.7.2	Show order status
1.7.3	Show rider location
1.7.4	Show estimated time
1.8.0	Seller Panel
1.8.1	Add product
1.8.1.1	Enter product details
1.8.1.2	Upload image
1.8.1.3	Set price
1.8.2	Manage products
1.8.3	Update stock
1.8.4	Receive order notification
1.8.5	Accept / reject order

1.9.0	Rider Panel
1.9.1	Receive delivery request
1.9.2	View pickup location
1.9.3	View drop location
1.9.4	Accept / reject job
1.9.5	Navigate using map
1.9.6	Update delivery status
1.10.0	Payment System
1.10.1	Select payment method
1.10.2	Process payment
1.10.3	Confirm transaction
1.10.4	Store payment history
1.11.0	Notification System
1.11.1	Send order notification
1.11.2	Send delivery updates
1.11.3	Send payment confirmation
1.11.4	Show real-time alerts
1.12.0	Feedback System

1.12.1	Give rating
1.12.2	Write review
1.12.3	Submit feedback
1.12.4	Display feedback

2.2.2 Non-Functional Requirements

NFR ID	Requirement Type	Description
NFR-01	Performance	The system shall load main pages such as dashboard, product search, cart, and order tracking quickly. Pages should load within 3 seconds under normal internet conditions.
NFR-02	Response Time	The system shall show search results for nearby shops and products within a short time. Product search results should appear within 2–4 seconds.
NFR-03	Availability	The GeoMart platform shall remain available for users most of the time. The system should maintain at least 99% uptime.
NFR-04	Reliability	The system shall correctly process orders, payments, stock updates, and delivery status without data loss. Order and payment records must be saved successfully after every transaction.
NFR-05	Security	The system shall protect user accounts using secure login, password validation, and OTP verification. Unauthorized users must not access buyer, seller, rider, or admin data.

NFR-06	Data Privacy	The system shall protect personal information such as name, phone number, email, address, and payment history. User data must not be visible to other unauthorized users.
NFR-07	Usability	The system shall provide a simple and user-friendly interface for buyers, sellers, and riders. New users should be able to register, search, order, or accept jobs without confusion.
NFR-08	Scalability	The system shall support an increasing number of users, shops, products, and orders. The system should handle growth without major performance issues.
NFR-09	Compatibility	The system shall work properly on common Android devices, iOS devices, and modern web browsers. Core features must function on Chrome, Firefox, Edge, Android, and iOS.
NFR-10	Maintainability	The system shall be designed in a modular way so that developers can update features easily. Changes in buyer, seller, or rider modules should not break the whole system.
NFR-11	Real-Time Update	The system shall update stock, order status, notifications, and rider location in real time. Buyers, sellers, and riders should see updated information instantly or within a few seconds.
NFR-12	Accuracy	The system shall show accurate product availability, shop distance, delivery time estimate, and payment history. Displayed information must match the latest database records.
NFR-13	Backup and Recovery	The system shall keep backup copies of important data such as orders, users, products, payments, and reviews. Data should be recoverable if a system failure occurs.
NFR-14	Accessibility	The system shall use readable text, clear buttons, and simple navigation. Users should be able to operate the system with minimum effort.

NFR-15	Portability	The system shall be flexible enough to be deployed on different servers or cloud platforms. The application should run after deployment without major code changes.
NFR-16	Integrity	The system shall prevent incorrect or duplicate records for orders, payments, inventory, and delivery jobs. The same order or payment should not be processed twice accidentally.
NFR-17	Notification Efficiency	The system shall send important notifications quickly to buyers, sellers, and riders. Order, delivery, and payment notifications should be delivered within a few seconds.
NFR-18	Auditability	The system shall keep records of important user actions such as orders, payments, stock updates, and delivery status changes. Admin or authorized users should be able to review transaction history when needed.
NFR-19	Fault Tolerance	The system shall handle errors such as failed payments, unavailable riders, or network failure gracefully. Users should receive clear error messages and the system should not crash.
NFR-20	Localization	The system shall support local usage needs such as nearby shops, local addresses, local payment methods, and local delivery areas. Users should be able to search and order from shops within their nearby location.

3.1.2 Class Diagram

The Class Diagram represents the structural design of the GeoMart system by showing the main classes, attributes, methods, and relationships between buyers, sellers, riders, products, orders, and payments.

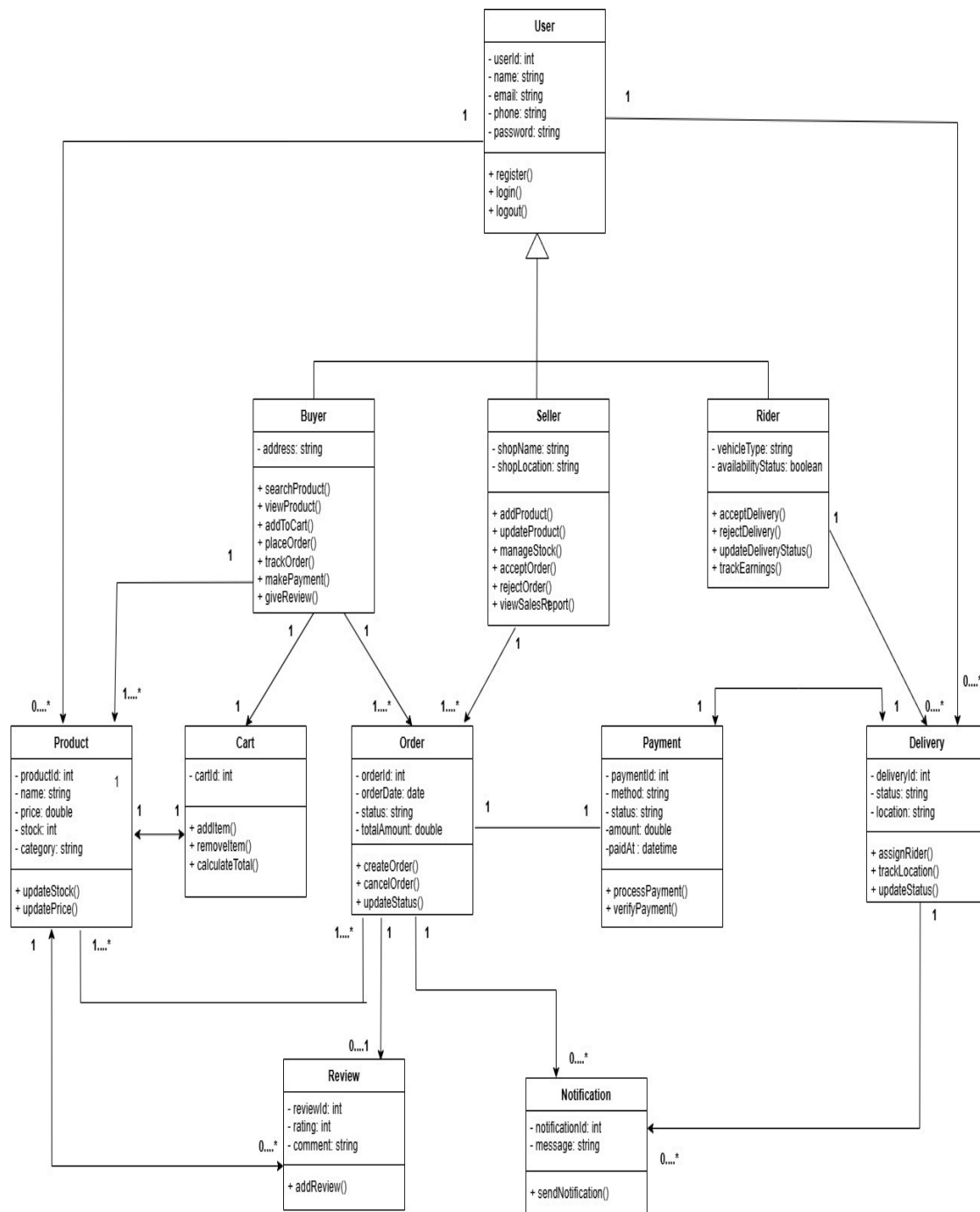


Fig 4: Class Diagram of GeoMart System

3.1.3 Activity Diagram

The Activity Diagram illustrates the workflow of the GeoMart system, including product searching, order placement, payment processing, seller confirmation, rider delivery, and order completion process.

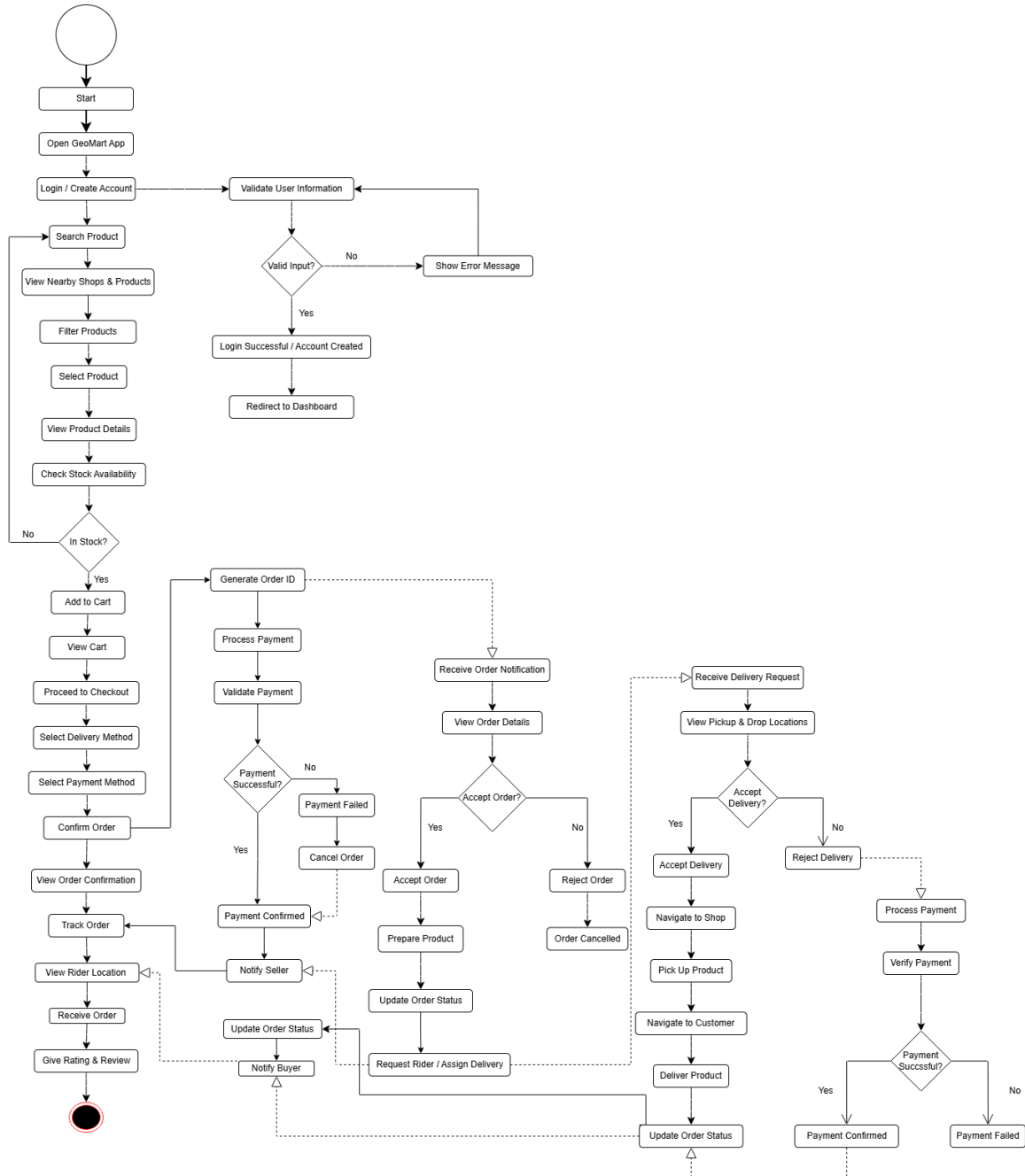


Fig 5: Activity Diagram of GeoMart System

3.2 Wireframe Design

The wireframe designs of GeoMart were created using Figma based on the functional requirements and user stories defined in the SRS. The prototype demonstrates the interaction flow between buyers, sellers, and riders.

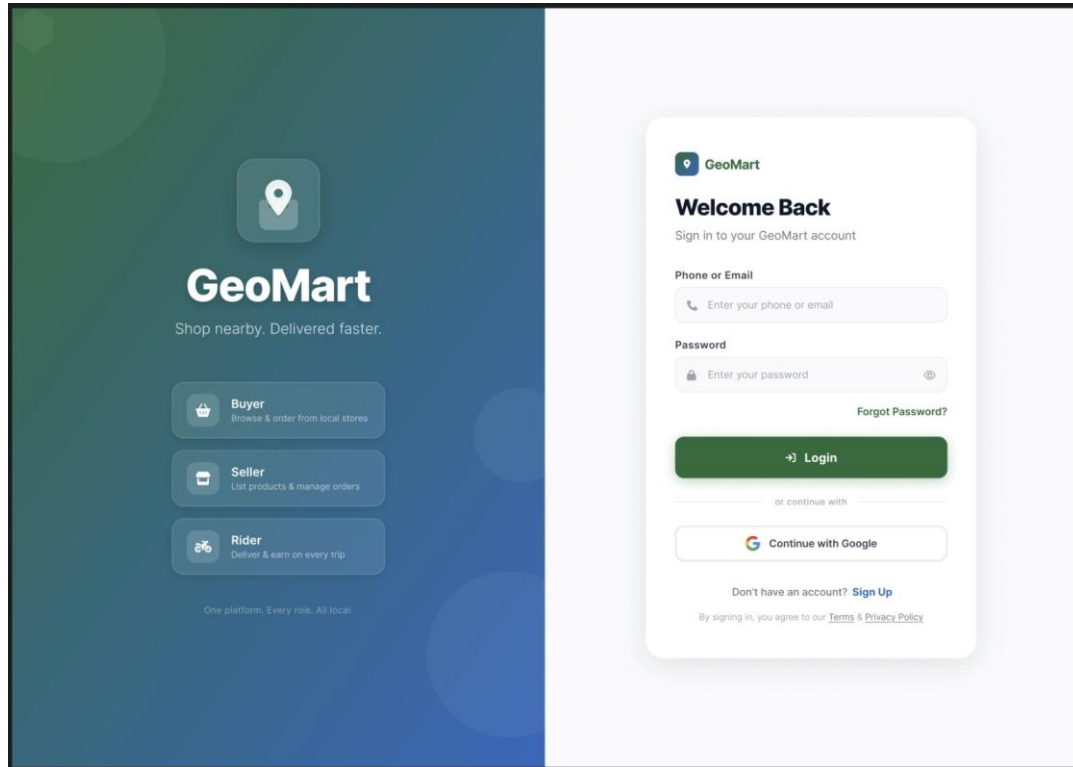


Fig 6: GeoMart User Login Interface Design

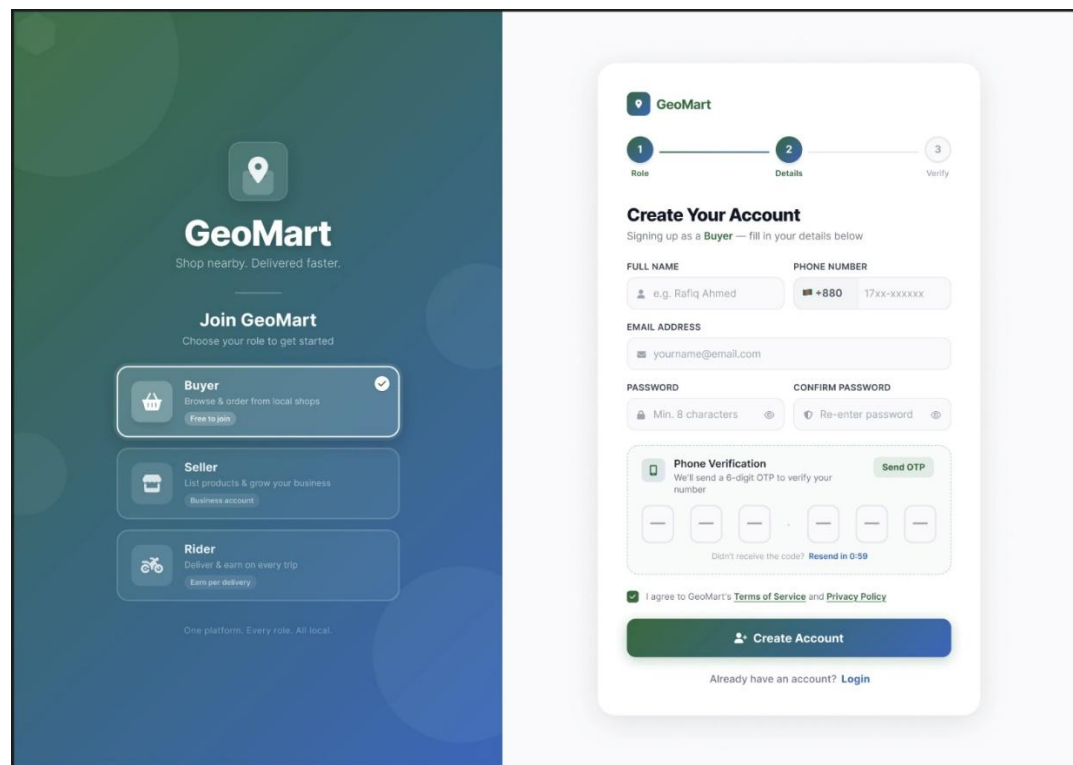


Fig 7: GeoMart User Registration and Verification Interface

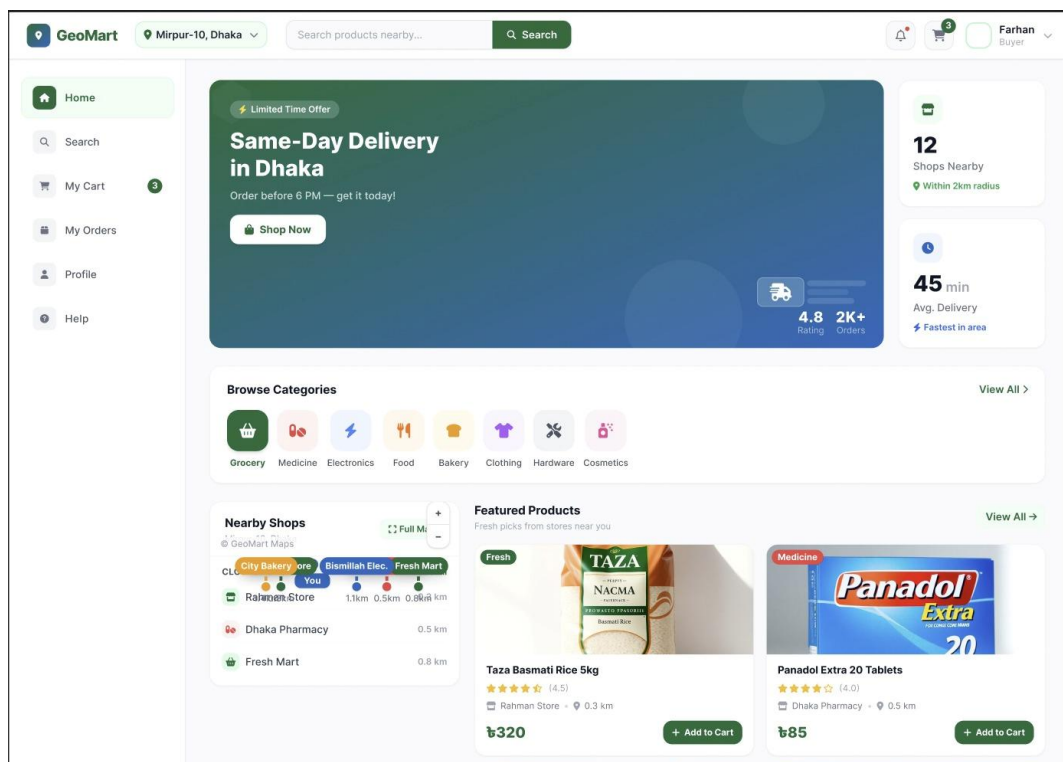


Fig 8: GeoMart Buyer Dashboard and Homepage Design

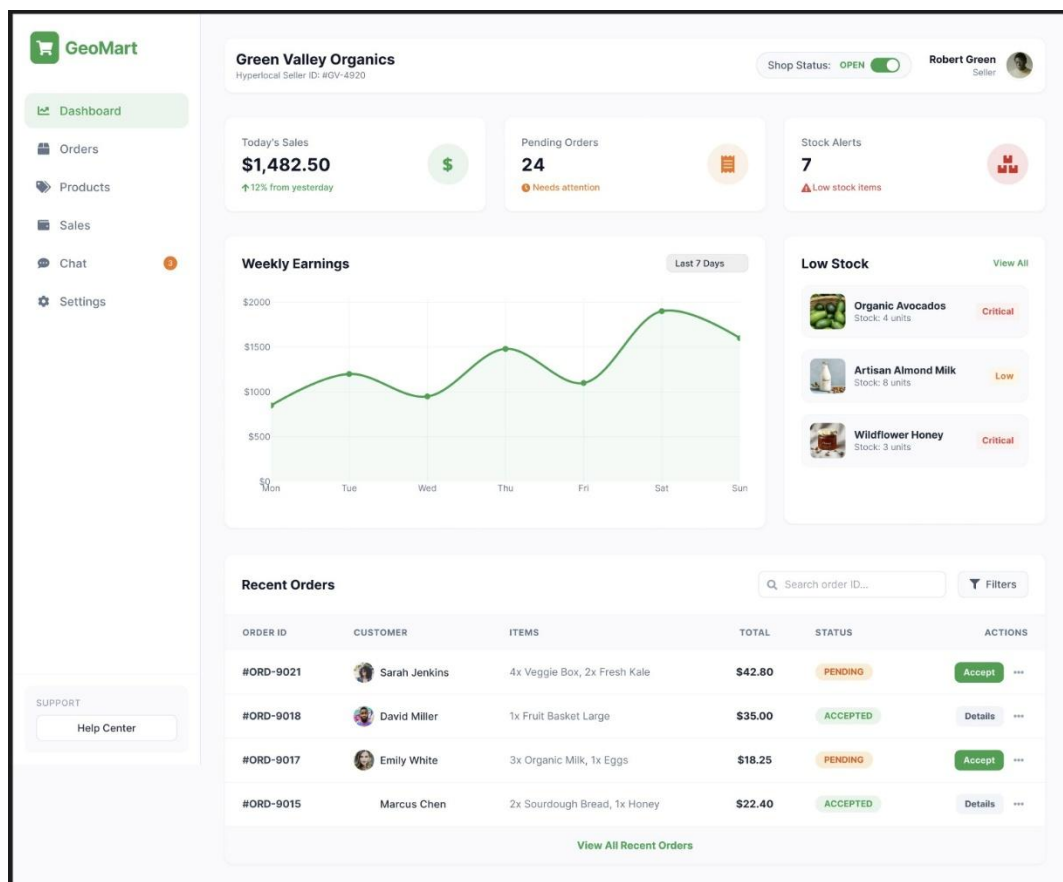


Fig 9: GeoMart Seller Dashboard Interface Design

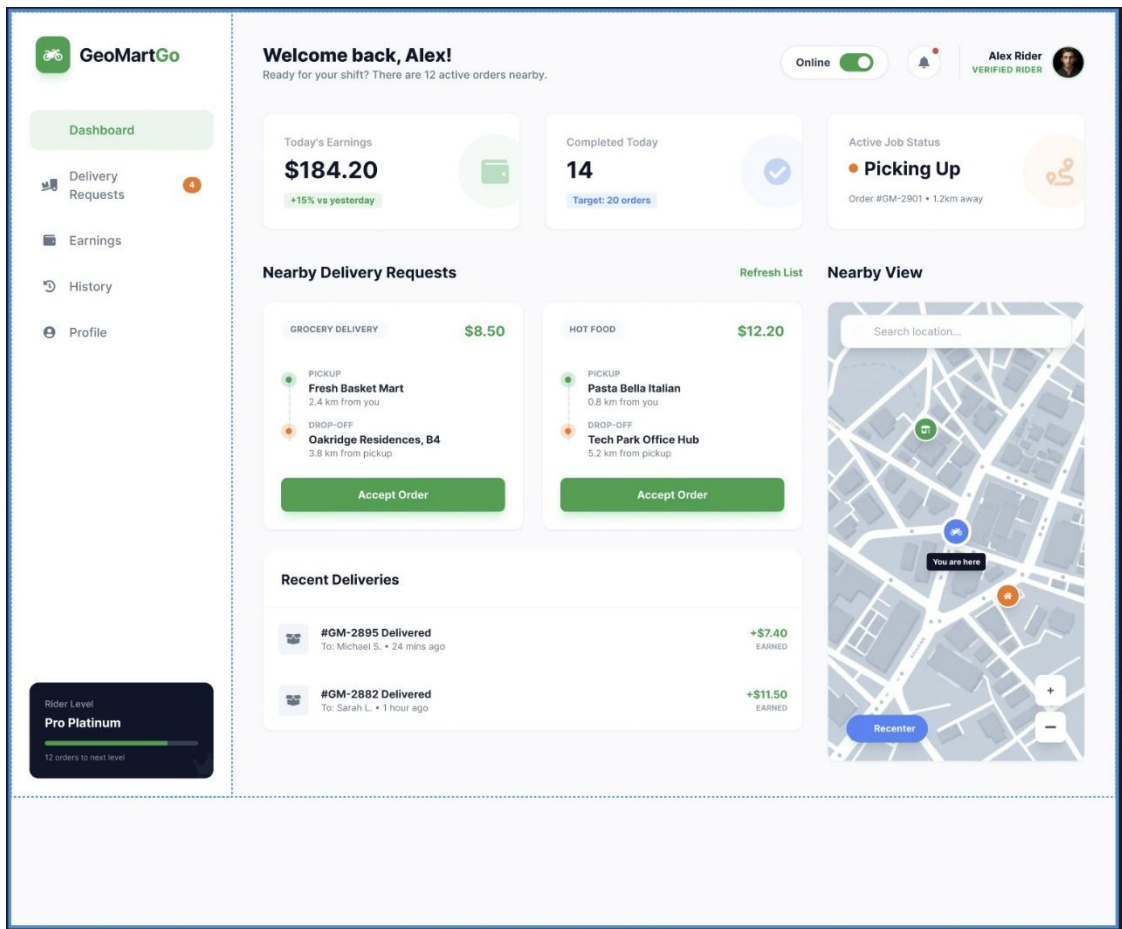


Fig 10: GeoMart Rider Delivery Management Dashboard Design

4. GIT WORKFLOW

The Git workflow of GeoMart was managed using GitHub branches and collaborative version control practices.

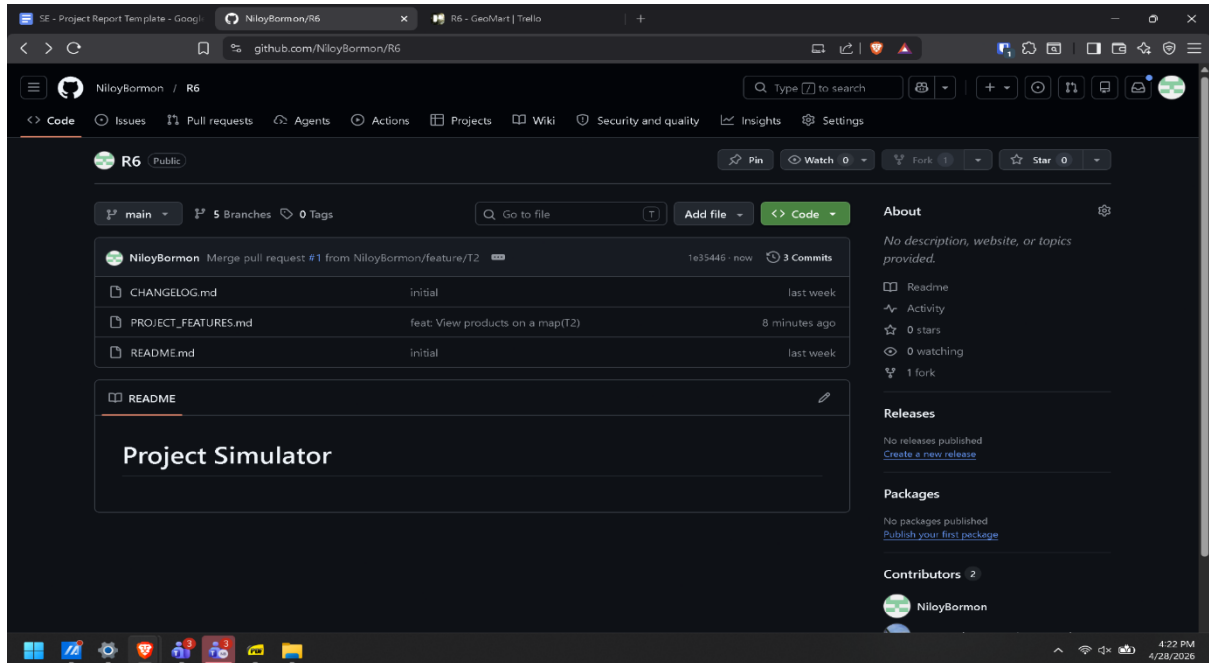


Fig 11: Main Repository Branch of GeoMart (Niloy Bormon)

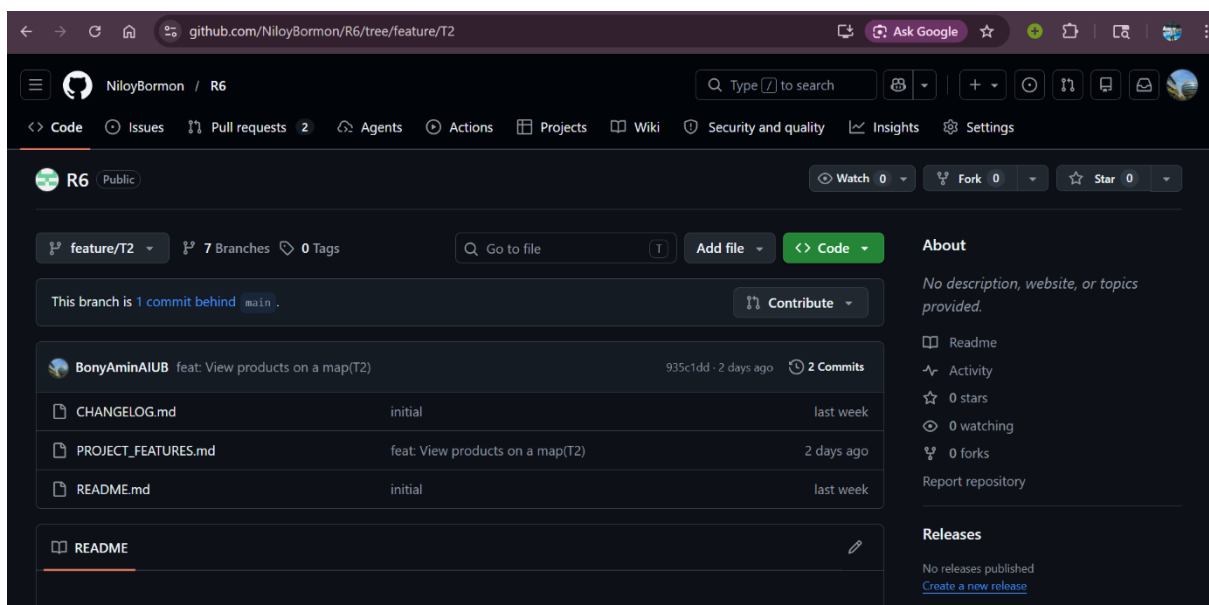


Fig 12: Sub Repository Branch of GeoMart (Md Bony Amin)

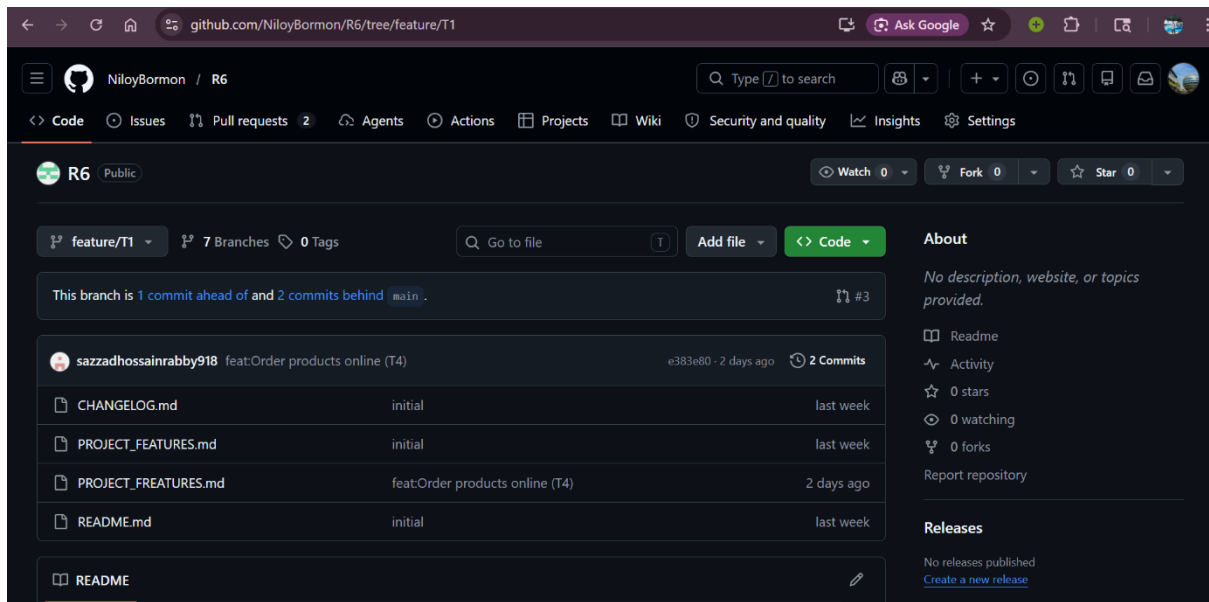


Fig 13: Sub Repository Branch of GeoMart (Sazzad Hossain Rabby)

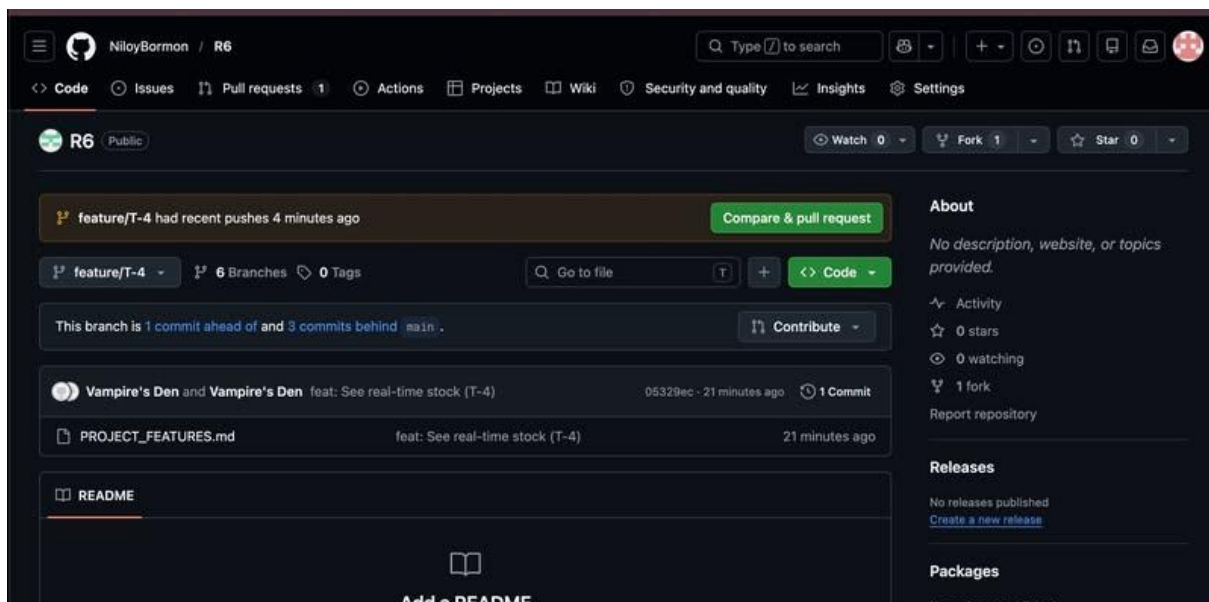


Fig 14: Sub Repository Branch of GeoMart (Abdullah Ar Rafi)

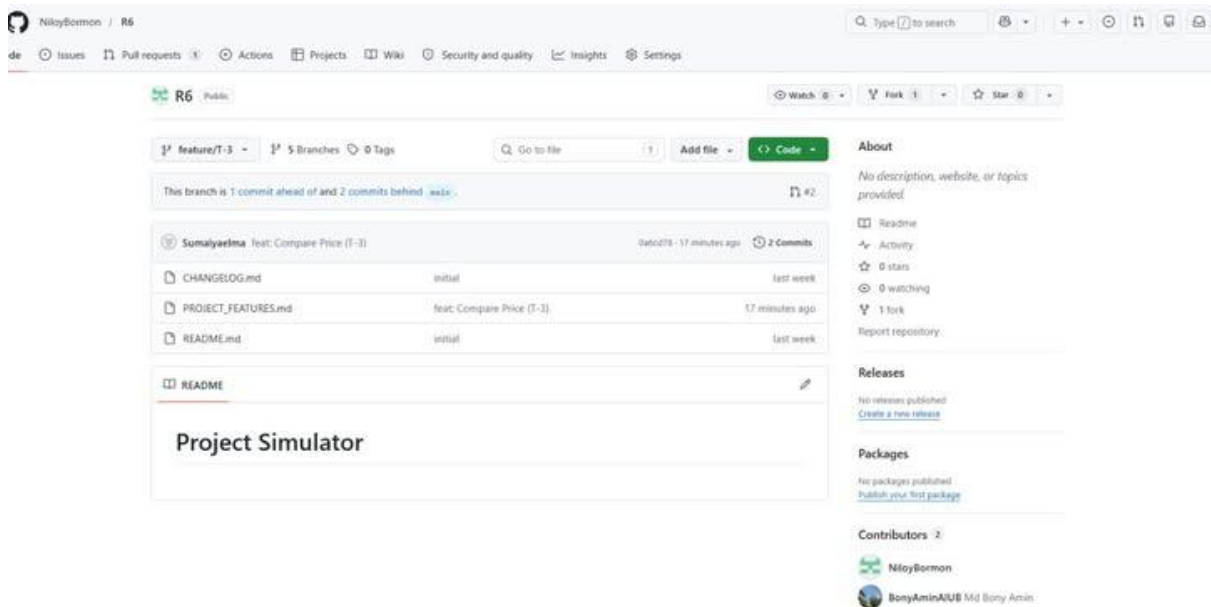


Fig 15: Sub Repository Branch of GeoMart (Sumaiya Elma Eshara)

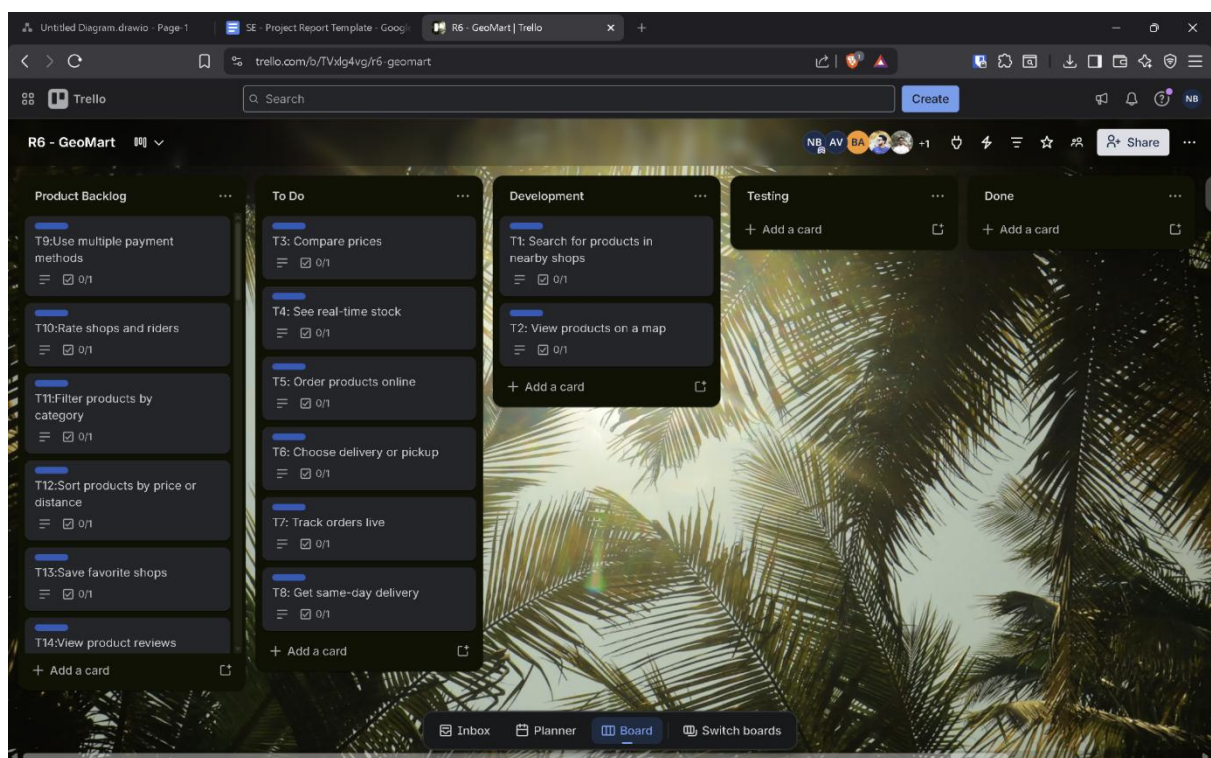


Fig 16: GeoMart Trello Board - Initial Product Backlog and Sprint Tasks

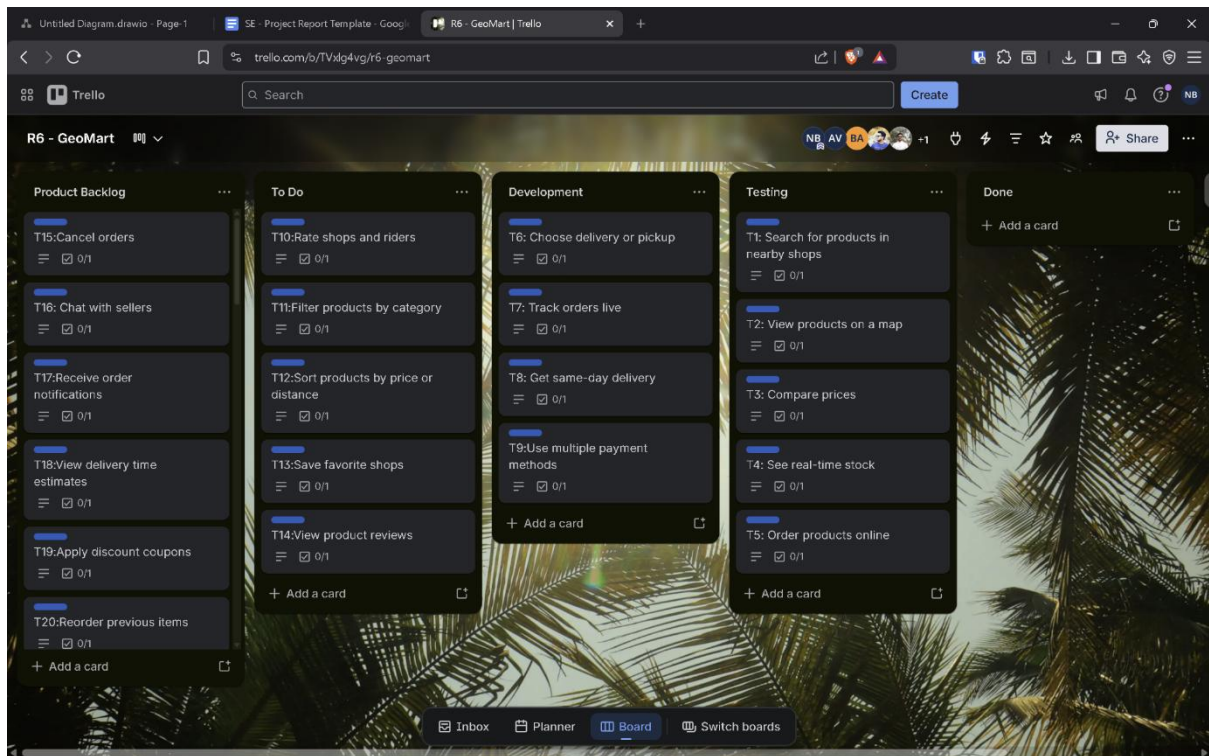


Fig 17: GeoMart Trello Board - Task Distribution and Testing Progress

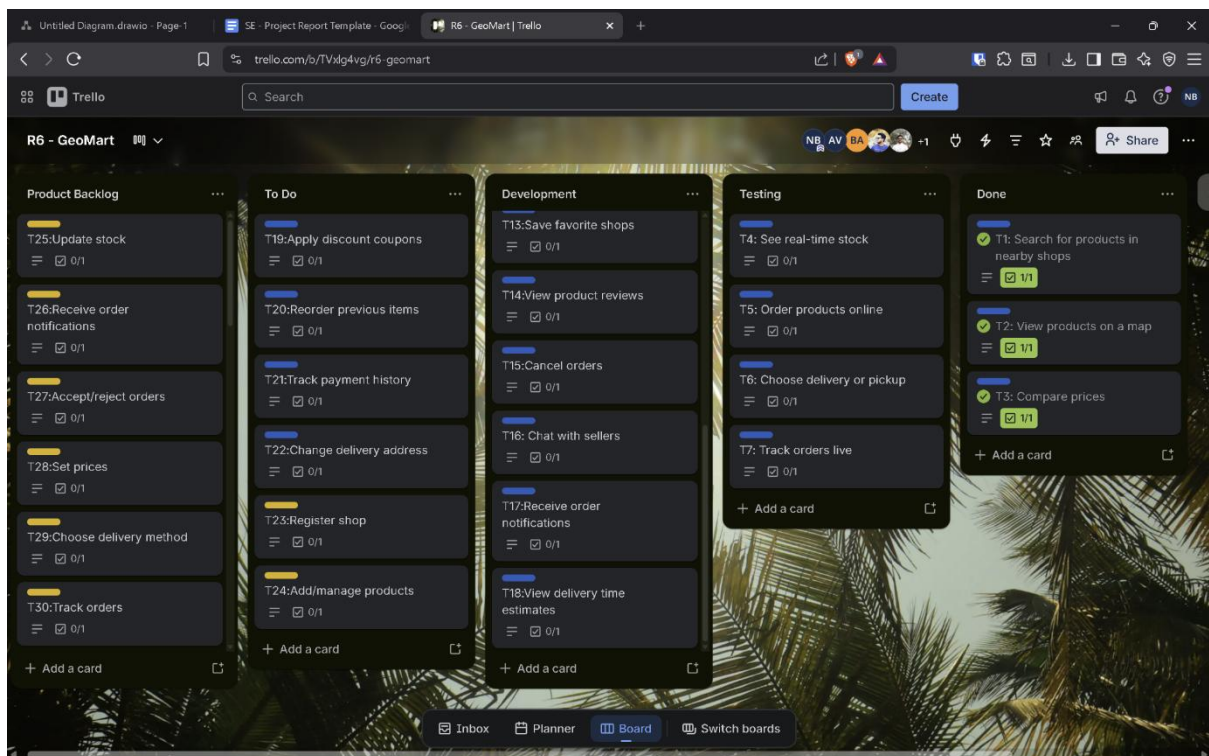


Fig 18: GeoMart Trello Board - Completed Tasks and Workflow Status

5. SOFTWARE TESTING

1. Verify user registration with valid details

Project Name: GeoMart			Test Designed by: Md Bony Amin	
Test Case ID: GM_TC_01			Test Designed date: 3/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Md Bony Amin	
Module Name: GeoMart App Registration			Test Execution date: 3/5/2026	
Test Title: Verify user registration with valid details				
Description: Test the user registration functionality				
Precondition: User is on the registration page				
Dependencies: Internet connection, Database, OTP Service				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open GeoMart app 2. Click Sign Up 3. Select Buyer role 4. Enter user information 5. Enter OTP 6. Click Register	Name: Bony Amin Username: bony Phone: 01625596262 Email: 24-56311-1@student.aiub.edu Password: 123456	User account should be created successfully and redirected to dashboard/login page	As expected	Pass
Post Condition: New buyer account is stored in database successfully.				

2. Verify successful login with valid credentials

Project Name: GeoMart	Test Designed by: Md Bony Amin
-----------------------	--------------------------------

Test Case ID: GM_TC_02			Test Designed date: 3/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Md Bony Amin	
Module Name: Login			Test Execution date: 3/5/2026	
Test Title: Verify login with valid credentials				
Description: Test login functionality using valid username and password				
Precondition: User must have a registered account				
Dependencies: Internet connection, Database				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open login page 2. Enter email 3. Enter password 4. Click Login	Username: bony Password: 123456	User should login successfully and open dashboard	As expected	Pass
Post Condition: User session is created successfully.				

3. Verify product search in nearby shops

Project Name: GeoMart		Test Designed by: Niloy Bormon		
Test Case ID: GM_TC_03		Test Designed date: 4/5/2026		
Test Priority (Low, Medium, High): High		Test Executed by: Niloy Bormon		
Module Name: Product Search		Test Execution date: 4/5/2026		
Test Title: Verify product search in nearby shops				

Description: Test searching products from nearby shops				
Precondition: Buyer must be logged in and location enabled				
Dependencies: GPS, Internet connection, Database				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open dashboard 2. Click search field 3. Enter product name 4. Click search	Product: Rice	Nearby shops with the searched product should be displayed successfully.	As expected	Pass
Post Condition: Product search history is stored successfully.				

4. Verify product filtering and sorting functionality

Project Name: GeoMart			Test Designed by: Niloy Bormon	
Test Case ID: GM_TC_04			Test Designed date: 4/5/2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: Niloy Bormon	
Module Name: Product Filter & Sort			Test Execution date: 4/5/2026	
Test Title: Verify filter and sorting functionality				
Description: Test filtering and sorting products				
Precondition: Product search results must be available				
Dependencies: Internet connection, Database				
Test Steps	Test Data	Expected Results	Actual Results	Status

1. Search product 2. Apply category filter 3. Sort by price 4. Sort by distance	Category: Grocery	Correct filtered and sorted product list should appear	As expected	Pass
Post Condition: Filtered product list is displayed correctly.				

5. Verify successful order placement

Project Name: GeoMart			Test Designed by: Sazzad Hossain Rabby	
Test Case ID: GM_TC_05			Test Designed date: 4/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Sazzad Hossain Rabby	
Module Name: Order System			Test Execution date: 4/5/2026	
Test Title: Verify placing an order successfully				
Description: Test buyer order placement process				
Precondition: Buyer must be logged in and cart must contain products				
Dependencies: Payment Gateway, Database				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Add product to cart 2. Open cart 3. Proceed checkout 4. Confirm order	Product: Oil Quantity: 2	Order should be placed successfully	As expected	Pass
Post Condition: Order information is saved in database.				

6. Verify live order tracking system

Project Name: GeoMart			Test Designed by: Sazzad Hossain Rabby	
Test Case ID: GM_TC_06			Test Designed date: 4/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Sazzad Hossain Rabby	
Module Name: Tracking			Test Execution date: 4/5/2026	
Test Title: Verify live order tracking				
Description: Test tracking delivery status and rider location				
Precondition: Order must already be confirmed				
Dependencies: GPS, Internet connection				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open orders 2. Select active order 3. Click Track Order	Order ID: 1005	Rider location and delivery status should appear correctly	As expected	Pass
Post Condition: Tracking information is updated in real-time.				

7. Verify seller product addition functionality

Project Name: GeoMart			Test Designed by: Abdullah Ar Rafi	
Test Case ID: GM_TC_07			Test Designed date: 6/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Abdullah Ar Rafi	

Module Name: Seller Product Management			Test Execution date: 6/5/2026	
Test Title: Verify seller product addition				
Description: Test adding products by seller				
Precondition: Seller must be logged in				
Dependencies: Database, Image Upload Service				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open seller dashboard 2. Click Add Product 3. Enter product details 4. Upload image 5. Save product	Product: Sugar Price: 120	Product should be added successfully	As expected	Pass
Post Condition: Product details are saved in inventory database.				

8. Verify product stock update functionality

Project Name: GeoMart	Test Designed by: Abdullah Ar Rafi
Test Case ID: GM_TC_08	Test Designed date: 6/5/2026
Test Priority (Low, Medium, High): Medium	Test Executed by: Abdullah Ar Rafi
Module Name: Stock Update	Test Execution date: 6/5/2026
Test Title: Verify product stock update	

Description: Test updating available stock quantity				
Precondition: Seller product must exist in inventory				
Dependencies: Database				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open inventory 2. Select product 3. Change stock quantity 4. Save update	Product: Sugar Stock: 50	Updated stock quantity should appear instantly	As expected	Pass
Post Condition: Inventory stock is updated successfully.				

9. Verify rider delivery request notification

Project Name: GeoMart			Test Designed by: Sumaiya Elma Eshara	
Test Case ID: GM_TC_09			Test Designed date: 7/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Sumaiya Elma Eshara	
Module Name: Rider Delivery Request			Test Execution date: 7/5/2026	
Test Title: Verify rider receives delivery request				
Description: Test rider delivery notification system				
Precondition: Order must be confirmed by seller				
Dependencies: Notification Service, GPS				
Test Steps	Test Data	Expected Results	Actual Results	Status

1. Login as rider 2. Wait for delivery request 3. Open notification	Delivery ID: D120	Rider should receive nearby delivery request instantly	As expected	Pass
Post Condition: Delivery request is assigned successfully.				

10. Verify payment confirmation process

Project Name: GeoMart			Test Designed by: Sumaiya Elma Eshara	
Test Case ID: GM_TC_10			Test Designed date: 7/5/2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: Sumaiya Elma Eshara	
Module Name: Payment System			Test Execution date: 7/5/2026	
Test Title: Verify payment confirmation process				
Description: Test payment system with mobile banking				
Precondition: Buyer must proceed to checkout page				
Dependencies: Payment Gateway, Internet connection				
Test Steps	Test Data	Expected Results	Actual Results	Status
1. Open checkout 2. Select payment method 3. Confirm payment	Payment Method: bKash Amount: 500 BDT	Payment should be processed and confirmed successfully	As expected	Pass
Post Condition: Payment history is stored successfully in database.				

6. CONCLUSION

GeoMart is a smart hyperlocal e-commerce platform designed to connect buyers, sellers, and riders within the same local area. The main goal of this project is to reduce delivery time and support local businesses by allowing customers to search and purchase products from nearby shops. Through features such as real-time product search, order tracking, secure payment methods, seller inventory management, and rider delivery services, the system creates a fast and efficient shopping experience for users.

During the development process, the Scrum (Agile) model was selected because it supports incremental development, teamwork, flexibility, and continuous improvement. Different system designs, user stories, wireframes, functional requirements, non-functional requirements, and testing methods were prepared to ensure that the system meets user expectations properly.

GeoMart provides benefits for all three user groups by helping buyers receive products quickly, sellers expand their businesses digitally, and riders earn income through local deliveries. Overall, this project demonstrates how technology can improve local commerce and create a reliable, user-friendly, and scalable delivery ecosystem for modern communities.

The project also highlights the importance of modern software engineering practices in building efficient and scalable online marketplace solutions.

Future improvements may include AI-based product recommendations, live traffic-aware rider routing, and advanced analytics for sellers.